

CAPSTONE PROJECT · ITERATION & INCREMENT PLAN

Mira: AI-Powered Project Intelligence Assistant

Iteration & Increment Plan

This plan describes how Mira is built incrementally — one working slice at a time, each proven before the next is added. It follows the course principle of "get one flow working end-to-end first, then expand," and maps each increment to the phases the author will demonstrate.

Prepared by: Praveen Kumar Jatta — Technical Program Management

Program: Applied Agentic AI for PMs/TPMs · Capstone: Mira

Method: Incremental build — vertical slices, each validated against ground truth

Date: Friday, August 7th, 2026

Document: 3 of 4 — BRD · PRD · Iteration Plan · Ground-Truth Dataset

Contents

Section	Page
1. Build Philosophy	3
2. The Increments (v0 → v5)	3
3. Phase Mapping & Submission Timeline	5
4. Definition of Done per Increment	6
5. Role Sign-offs (Solo SDLC)	6

1. Build Philosophy

Mira is built in **vertical slices**, not all-at-once. Each increment delivers one working capability end-to-end — request in, grounded output out — and is validated before the next is added. This is the course's central build rule: *get one flow working end-to-end first, then expand one agent at a time*.

Why incremental: building all agents at once produces a "black box" that is impossible to debug. Building one slice at a time means every failure is isolated, every fix is measurable, and the vague-input guardrail is proven on each agent before the next is introduced.

Three rules hold across every increment:

- **Guardrail from day one** — the "if vague → ask / UNKNOWN" rule is in every agent's prompt from its first version, not added later.
- **Trace early** — observability (Langfuse / n8n logs) is connected at v1, so every increment is measurable.
- **Prove before proceeding** — an increment isn't "done" until its relevant test cases pass against the ground-truth dataset.

2. The Increments (v0 → v5)

v0 — Skeleton & Grounding Harness

Goal: prove data flows in and out before any intelligence is added.

- Set up the n8n workflow shell and a manual trigger
- Load the four data files (description, timeline CSV, risk CSV, task board CSV) into the workflow
- Confirm the raw data can be read and passed to an LLM node

Proven when: a trivial prompt returns data drawn from the CSVs (e.g., "how many tasks are in the board?" → 25).

v1 — First Capability: Planner (end-to-end)

Goal: one complete slice — the hardest-value capability working, with the guardrail.

- Build the Planner agent with the five-part prompt + grounding + vague-input guardrail
- Connect observability (Langfuse / n8n execution logs)
- Ground the Planner in the description + timeline CSV

Proven when: T1 produces a grounded 8-phase plan, AND T2 & T9 (vague) correctly ask for more information instead of fabricating a plan.

v2 — Add the Router + Risk Assessor

Goal: introduce the routing pattern and the second specialist.

- Add the Orchestrator / Router that classifies request type and dispatches
- Build the Risk Assessor agent grounded in the risk CSV
- Route "plan" vs "risk" requests to the correct agent

Proven when: T3 → categorized risk matrix; T7 → top 3 risks; T4 (vague) → asks for info; and the router sends each request to the right agent.

v3 — Add the Status Reporter

Goal: the third core capability, with correct counts and blocker detection.

- Build the Status Reporter grounded in the task board CSV
- Support sprint-scoped and overall status

Proven when: T5 → Sprint 3 status; T10 → correct counts (5 Done / 3 In Progress / 16 To Do / 1 Blocked); T6 (no data) → asks for data.

v4 — Add the Milestone Tracker (extended capability)

Goal: the extended capability — proactive at-risk / blocked detection.

- Build the Milestone Tracker comparing timeline vs task-board progress
- Correctly surface blocked tasks (T024) and upcoming milestones

Proven when: T8 → identifies T024 (Security review, Blocked); T11 → real upcoming milestones from the timeline.

v5 — Delivery & Integration

Goal: satisfy the business requirement that outputs are delivered, not just generated.

- Add the stakeholder-update generation (T12) grounded in the task board
- Wire the Gmail node to deliver the update via email
- Wire the Slack node to post the update / report

Proven when: T12 → a grounded Sprint 2 stakeholder update, delivered to both Gmail and Slack.

3. Phase Mapping & Submission Timeline

The increments map to the capstone phases. For this submission, Phases 1–3 are delivered first (Sunday); Phases 4–5 follow the next week, alongside the Langfuse coursework.

Phase	Increments	Deliverable	Target
Phase 1 — Plan	—	BRD, PRD, Iteration Plan, Ground-Truth Dataset	Fri–Sat
Phase 2 — Build	v0 → v5	Working n8n Mira workflow (all 4 agents + delivery)	Fri–Sat
Phase 3 — Baseline Test	—	All 12 inputs (T1–T12) run once, results documented vs ground truth	Sat–Sun
Phase 4 — Evaluate (next week)	—	~10 runs for consistency; evaluate → fix prompt → re-test; optional fine-tune	Next week
Phase 5 — Document (next week)	—	Architecture diagram, writeup, cost analysis, 3 metrics, slide deck, GitHub	Next week

Scope note: the mandatory pipeline is built in **n8n**. A secondary **LangFlow** pipeline (native Langfuse observability) is a bonus increment attempted only if time permits after Phase 3.

4. Definition of Done per Increment

An increment is "done" only when all four hold:

Criterion	Check
Grounded	Every fact in the output traces to the provided data
Guarded	The increment's vague test case(s) correctly ask for information
Traced	The run is visible in observability (input/output, tokens, cost)
Tested	The increment's mapped test cases pass against the ground-truth dataset

5. Role Sign-offs (Solo SDLC)

As a solo capstone, the author performs and documents every role. Each increment is signed off from the relevant role's perspective:

Role	Responsibility across increments
Product Owner	Confirms each increment delivers the intended capability and value
Business Stakeholder	Confirms outputs are trustworthy and useful for real decisions
Developer	Builds the agents, prompts, routing, and delivery integrations
Tester	Runs each increment's test cases against ground truth; logs pass/fail
Project Manager	Sequences increments, tracks the phase timeline, manages scope
End User	Validates that the output is what a PM/TPM would actually want and use

Documentation discipline: every process, decision, and design thought is recorded — in these four documents, in the test-results log, and in the final writeup — so the project is fully traceable on paper, as required.